From schema to knowledge graph

Manufacturing Hybrid GraphRAG — the exact path documents take between 'schema YAML saved' and 'answers cite graph nodes'. Six stages, four extractors, one schema as the source of truth.

0. The trigger

Either the Streamlit **Save & build KG** button or the CLI `python main.py --rebuild --domain <id> --no-llm`. Both end up in `pipeline/unified_pipeline.py:149` calling `ManufacturingPipeline(domain=<id>).build_or_load(rebuild=True)`.

Stage 1 — Ingestion

doc_pipeline/document_ingestion.py

```
doc_pipeline/input_docs/<domain>/**            <- what you staged

DocumentIngestion.ingest_directory()    (recursive rglob)
   per file -> ingest_file() -> parser by extension:
       .pdf   -> PDFParser   (pdfplumber: per-page text + tables)
       .txt   -> TXTParser   (whole file = one Document)
       .xlsx  -> ExcelParser (one Document per sheet)

list[Document]  with .content + .metadata
   (page, sheet_name, source, classification, ...)
```

Folder name → `metadata.classification` via `core/document_acl.py:60`:

Subfolder segment	Classification
management/ or confidential/	confidential
restricted/ or internal/	restricted
anything else	public

Stage 2 — Chunking

doc_pipeline/chunking.py

```
HybridChunker.chunk_documents(documents)
   three strategies, picked per Document:
      recursive       (long prose, paragraph-aware split)
      semantic        (embedding-similarity boundary detection)
      sliding_window  (fallback for short texts)

list[Chunk]   with .text, .metadata, .chunk_id (int), .strategy
```

Chunks inherit the source Document's metadata (page, sheet, classification, etc.).

Stage 3 — Embedding → Qdrant

doc_pipeline/embeddings.py

```
EmbeddingPipeline(domain=<id>)
   bge-small-en-v1.5 encodes each chunk (384-dim, shared model cache)
   Qdrant upsert into collection `<domain>_corpus`
      (single Qdrant store, per-domain collection name)

save() writes:
```

```
doc_pipeline/vector_store/<domain>_index_chunks.json
doc_pipeline/vector_store/<domain>_index.manifest.json
```

Stage 4 — Adapter: chunks → core docs

`pipeline/adapter.py` **— this is where the schema's Equipment** `id_pattern` **drives extraction.**

```
chunks_to_core_docs(chunks, domain=<id>):
   for each chunk:
      derive stable chunk_id (md5 of source + index)
      regex-extract entity ids from chunk.text into metadata:
         equipment_ids  <- schema's Equipment id_pattern (anchored,
                           word-bound, stripped of ^$ for findall)
         alarm_codes    <- ALM-[A-Z]\d{3}
         part_numbers   <- SP-/TH-/BRK-/SFT-/HSG-/GR-\d{4}
         fault_codes    <- FC-\d{3}

list[{"chunk_id": "...", "text": "...", "metadata": {...}}]
```

Why this matters: when you author a clean `id_pattern` in your schema, asset-tag extraction is automatic. New domains' tags get lifted into `metadata.equipment_ids` without any Python edits.

Stage 5 — KG construction

`core/knowledge_graph.py` — `KnowledgeGraph(domain=<id>).build_from_documents(docs)` runs four extractors per chunk in priority order. Each extractor reads YOUR schema and emits Mentions + EdgeCandidates accordingly.

#	Extractor	Author tag	Conf.	Reads	What it emits
1	CodeExtractor	system:code	1.00	chunk text via every entity's id_pattern regex	Mentions for Equipment / Alarm / FailureMode / SparePart
2	MetadataExtractor	system:metadata	0.95	metadata.{equipment_ids, alarm_codes, part_numbers, fault_codes}	Mentions per declared in the schema (
3	KeywordExtractor	system:keyword	0.95	every entity_type with a 'vocabulary' block (Component, Symptom, etc.)	Whole-word, case-insensitive match + edges from m
4	NarrativeExtractor	system:llm_extract	0.50	regex over prose for open-vocab types	Low-confidence Symptom / Procedure mentions; HITL gap

Every Mention and EdgeCandidate goes through schema validation:

• `Schema.validate_entity(identifier, type_name)` — checks the `id_pattern` or `vocabulary` allow-list

• `Schema.validate_edge(relation, source_type, target_type)` — checks the edge is declared and endpoints match its source/target types

Rejects go into `kg._rejected` (inspectable for diagnosing schema drift). Accepted entries land in a NetworkX `DiGraph` with full provenance:

```
graph.add_node(identifier,
    entity_type="Component",
    chunk_ids={"chunk_42", "chunk_103"},
    provenance=Provenance(
        author="system:keyword",
        confidence=0.95,
        source_chunk_id="chunk_42",
        timestamp=1747408234.5,
        supersedes=None,         # set when HITL writes back
        notes="",
    ),
)

graph.add_edge("CV-301", "belt",
    relation="HAS_COMPONENT",
    weight=2,
    chunk_ids={...},
    provenance=Provenance(...),
```

```
)
```

Stage 6 — Persistence

After every node + edge is validated and stamped:

```
data/processed/knowledge_graph.<domain>.json     <- KG snapshot

{
  "nodes": {
    "CV-301": {"entity_type": "Equipment",
               "chunk_ids": [...],
               "provenance": {"author": "system:metadata",
                              "confidence": 0.95, ...}},
    "belt":   {"entity_type": "Component",
               "chunk_ids": [...],
               "provenance": {...}}
  },
  "edges": [
    {"source": "CV-301", "target": "belt",
     "relation": "HAS_COMPONENT",
     "weight": 2, "chunk_ids": [...],
     "provenance": {...}}
  ]
}
```

What gets queried at runtime

When a user types a question, the KG isn't queried for facts directly — it acts as a **retrieval bias**. Chunks linked to query-relevant entities get boosted; unrelated chunks get demoted. That's how the answer's symptom → cause → procedure chain stays grounded in evidence rather than LLM hallucination.

```
query
   ClarifierAgent -> extracts entity-typed slots
                     (equipment_id, metric, etc.)

   KnowledgeGraph.get_subgraph_for_query()
      - matches query entities to KG nodes
      - walks declared edges per the schema's `traversal_routes`
      - returns a subgraph + a chunk allow-list

   HybridRetriever (BM25 + Qdrant vector + RRF + cross-encoder reranker)
      - scores chunks
      - prefers chunks in the KG allow-list

   LLM (task_model("answer") per the router) generates the answer

   Critic + Guardrails verify citations against the allow-list
```

One-glance diagram

```
schemas/<domain>.yaml --------------------------------+
                                                      |
doc_pipeline/input_docs/<domain>/**                   |
   |                                                  |
   v DocumentIngestion (PDF / TXT / XLSX parsers)     |
list[Document]                                        |
   |                                                  |
   v HybridChunker                                    |
list[Chunk]                                           |
   |                                                  |
   +--> EmbeddingPipeline -> Qdrant                   |
   |       <domain>_corpus collection                 |
   |                                                  |
   v adapter.chunks_to_core_docs                      |  schema drives:
```

```
[{chunk_id, text, metadata}]                          |    EQUIPMENT_RE,
   |                                                  |    vocabularies,
   v KnowledgeGraph.build_from_documents              |    edge declarations
       4 extractors per chunk          <--------------+
       schema validates every Mention / Edge <-------+
       provenance stamped on every node / edge
       rejects go to kg._rejected
   |
   v
data/processed/knowledge_graph.<domain>.json
```

Two artifacts result: the Qdrant collection for similarity search, and the KG json for typed retrieval routing. Both are consumed at query time by `pipeline/unified_pipeline.diagnostic()`.